In [2]:
```
# Step 1 – Load required libraries and dataset

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score, precision_score, reca

print(" Libraries loaded successfully.")

# ---- Load dataset ----
# Step 1 – Load dataset (same folder as notebook)

import pandas as pd

# Just use the file name since it's in the same directory
df = pd.read_csv("placement_data.csv")

# Show info
print(" Dataset loaded successfully!")
print("Rows:", df.shape[0], " | Columns:", df.shape[1])
print("\n Column Names:")
print(df.columns.tolist())

# Display first 5 rows
df.head()
```

```
 Libraries loaded successfully.
 Dataset loaded successfully!
Rows: 851  | Columns: 13

 Column Names:
['SEX(M/F)', 'Stream', 'Xth', 'Board of Xth ', 'XIIth', 'Board of XII', 'BACKLO
G', 'Btech', 'Training Status', 'Placement Status', 'Campus Drive attended', 'Pla
cement Year', 'Origin']
```

Out[2]:

| | SEX(M/F) | Stream | Xth | Board of Xth | XIIth | Board of XII | BACKLOG | Btech | Training Status | Placer St |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | M | EEN | 67.00 | WBBSE | 65.0 | WBCHSE | 1 | 64.00 | 0 | |
| **1** | M | EEN | 77.20 | WBBSE | 77.8 | WBCHSE | 1 | 63.10 | 0 | |
| **2** | M | EEN | 70.00 | WBBSE | 66.0 | WBCHSE | 1 | 44.85 | 0 | |
| **3** | M | EEN | 73.71 | WBBSE | 72.8 | WBCHSE | 1 | 76.00 | 0 | |
| **4** | M | EEN | 87.18 | WBBSE | 82.8 | WBCHSE | 0 | 72.60 | 0 | |

In [5]:
```
# Step 2 – Data preprocessing (using 'Placement Status' as target)

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

#  Define target column
TARGET = "Placement Status"

# Separate features and target
X = df.drop(columns=[TARGET])
y = df[TARGET]

# Detect numeric and categorical columns
num_cols = X.select_dtypes(include=['number']).columns.tolist()
cat_cols = X.select_dtypes(exclude=['number']).columns.tolist()

print(" Feature detection done!")
print("Numeric columns:", num_cols)
print("Categorical columns:", cat_cols)

# Preprocessing pipelines
numeric_transformer = Pipeline([
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Combine numeric + categorical pipelines
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, num_cols),
    ('cat', categorical_transformer, cat_cols)
])

# Fit and transform data
X_transformed = preprocessor.fit_transform(X)

print("\n Data preprocessing complete!")
print("Transformed feature shape:", X_transformed.shape)
```

```
 Feature detection done!
Numeric columns: ['Xth', 'XIIth', 'BACKLOG', 'Btech', 'Training Status', 'Campus
Drive attended', 'Placement Year']
Categorical columns: ['SEX(M/F)', 'Stream', 'Board of Xth ', 'Board of XII', 'Ori
gin']

 Data preprocessing complete!
Transformed feature shape: (851, 53)
```

In [6]:
```
# Step 3 – Implement the HTER (Hyperbolic Tangent Exponential Regression) model

import numpy as np
from scipy.optimize import minimize

class HTER:
    def __init__(self, alpha=1.0, gamma=0.1, max_iter=1000):
        self.alpha = alpha
        self.gamma = gamma
```

```
        self.max_iter = max_iter

    def _link(self, eta):
        """HTER link function."""
        z = self.alpha * eta * np.exp(-self.gamma * np.abs(eta))
        return 0.5 * (1 + np.tanh(z))

    def _neg_log_likelihood(self, beta, X, y):
        """Negative log-likelihood for HTER."""
        eta = X @ beta
        p = self._link(eta)
        # Add small epsilon to prevent log(0)
        eps = 1e-9
        ll = y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps)
        return -np.sum(ll)

    def fit(self, X, y):
        """Fit model by minimizing negative log-likelihood."""
        n_features = X.shape[1]
        beta_init = np.zeros(n_features)

        result = minimize(self._neg_log_likelihood, beta_init,
                          args=(X, y),
                          method='BFGS',
                          options={'maxiter': self.max_iter})

        self.beta = result.x
        return self

    def predict_proba(self, X):
        """Predict probabilities."""
        eta = X @ self.beta
        return self._link(eta)

    def predict(self, X, threshold=0.5):
        """Predict class labels."""
        return (self.predict_proba(X) >= threshold).astype(int)
```

In [7]:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_transformed, y, test_size=

# Initialize and train HTER model
hter = HTER(alpha=1.0, gamma=0.1)
hter.fit(X_train, y_train)

# Predictions
y_pred = hter.predict(X_test)
y_prob = hter.predict_proba(X_test)

# Evaluate
print(" Model training complete!\n")
print(" Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\n Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\n Classification Report:\n", classification_report(y_test, y_pred))
```

```
Model training complete!

Accuracy: 0.6959

Confusion Matrix:
[[77 26]
[26 42]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.75      0.75       103
           1       0.62      0.62      0.62        68

    accuracy                           0.70       171
   macro avg       0.68      0.68      0.68       171
weighted avg       0.70      0.70      0.70       171
```

In [9]:

```python
# Step 4 – Compare HTER with other classification models

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

# Define models
models = {
    "HTER (Proposed Model)": hter,
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "LDA": LinearDiscriminantAnalysis(),
    "SVM (RBF)": SVC(kernel="rbf", probability=True, random_state=42)
}

# Train and evaluate each model
results = []

for name, model in models.items():
    if name == "HTER (Proposed Model)":
        y_pred = model.predict(X_test)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    results.append({"Model": name, "Accuracy": acc})

# Convert results to DataFrame
results_df = pd.DataFrame(results).sort_values(by="Accuracy", ascending=False)

print(" Model Comparison Complete!\n")
print(results_df)

# Detailed performance for each model
print("\n Detailed Reports:")
```

```
for name, model in models.items():
    if name == "HTER (Proposed Model)":
        y_pred = model.predict(X_test)
    else:
        y_pred = model.predict(X_test)
    print(f"\n=== {name} ===")
    print(classification_report(y_test, y_pred))
```

```
 Model Comparison Complete!

                  Model  Accuracy
5             SVM (RBF)  0.713450
0  HTER (Proposed Model)  0.695906
1    Logistic Regression  0.695906
2             KNN (k=5)  0.678363
4                   LDA  0.672515
3         Decision Tree  0.625731

 Detailed Reports:

=== HTER (Proposed Model) ===
              precision    recall  f1-score   support

           0       0.75      0.75      0.75       103
           1       0.62      0.62      0.62        68

    accuracy                           0.70       171
   macro avg       0.68      0.68      0.68       171
weighted avg       0.70      0.70      0.70       171


=== Logistic Regression ===
              precision    recall  f1-score   support

           0       0.75      0.74      0.75       103
           1       0.61      0.63      0.62        68

    accuracy                           0.70       171
   macro avg       0.68      0.69      0.68       171
weighted avg       0.70      0.70      0.70       171


=== KNN (k=5) ===
              precision    recall  f1-score   support

           0       0.73      0.75      0.74       103
           1       0.60      0.57      0.59        68

    accuracy                           0.68       171
   macro avg       0.66      0.66      0.66       171
weighted avg       0.68      0.68      0.68       171


=== Decision Tree ===
              precision    recall  f1-score   support

           0       0.69      0.68      0.69       103
           1       0.53      0.54      0.54        68

    accuracy                           0.63       171
   macro avg       0.61      0.61      0.61       171
weighted avg       0.63      0.63      0.63       171


=== LDA ===
              precision    recall  f1-score   support

           0       0.73      0.72      0.73       103
```

```
              1       0.59      0.60      0.59        68

    accuracy                           0.67       171
   macro avg       0.66      0.66      0.66       171
weighted avg       0.67      0.67      0.67       171


=== SVM (RBF) ===
              precision    recall  f1-score   support

           0       0.76      0.76      0.76       103
           1       0.64      0.65      0.64        68

    accuracy                           0.71       171
   macro avg       0.70      0.70      0.70       171
weighted avg       0.71      0.71      0.71       171
```

In [11]:

```python
# Step 4A – Performance comparison of classifiers
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticD
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

# Assuming df already loaded and cleaned
# Encode categorical columns again if needed
df_encoded = df.copy()
for col in df_encoded.select_dtypes(include=['object']).columns:
    df_encoded[col] = LabelEncoder().fit_transform(df_encoded[col])

# Split features and target
X = df_encoded.drop("Placement Status", axis=1)
y = df_encoded["Placement Status"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define models
models = {
    "Logistic Regression": LogisticRegression(max_iter=500),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "LDA": LinearDiscriminantAnalysis(),
    "QDA": QuadraticDiscriminantAnalysis(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel="rbf", probability=True, random_state=42)
}

# Train & evaluate models
results = []
for name, model in models.items():
    model.fit(X_train, y_train)
```

```
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    f1 = f1_score(y_test, preds)
    results.append((name, acc, f1))

# HTER model (already implemented previously)
from scipy.optimize import minimize
import numpy as np

def hter_loss(params, X, y):
    n_features = X.shape[1]
    beta = params[:n_features]
    alpha, gamma = params[-2], params[-1]
    eta = X @ beta
    z = alpha * eta * np.exp(-gamma * np.abs(eta))
    p = 0.5 * (1 + np.tanh(z))
    eps = 1e-9
    return -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))

params0 = np.concatenate([np.zeros(X_train.shape[1]), [1, 0.1]])
opt = minimize(hter_loss, params0, args=(X_train, y_train), method='L-BFGS-B', b
params_hat = opt.x
beta_hat, alpha_hat, gamma_hat = params_hat[:-2], params_hat[-2], params_hat[-1]
eta_test = X_test @ beta_hat
z_test = alpha_hat * eta_test * np.exp(-gamma_hat * np.abs(eta_test))
p_test = 0.5 * (1 + np.tanh(z_test))
preds_hter = (p_test >= 0.5).astype(int)
acc_hter = accuracy_score(y_test, preds_hter)
f1_hter = f1_score(y_test, preds_hter)

results.append(("HTER", acc_hter, f1_hter))

# Display results
import pandas as pd
results_df = pd.DataFrame(results, columns=["Model", "Accuracy", "F1 Score"])
print(results_df)
```

```
                Model  Accuracy  F1 Score
0  Logistic Regression  0.736842  0.621849
1                  KNN  0.684211  0.571429
2                  LDA  0.719298  0.606557
3                  QDA  0.707602  0.576271
4        Decision Tree  0.707602  0.615385
5                  SVM  0.748538  0.638655
6                 HTER  0.701754  0.622222
```

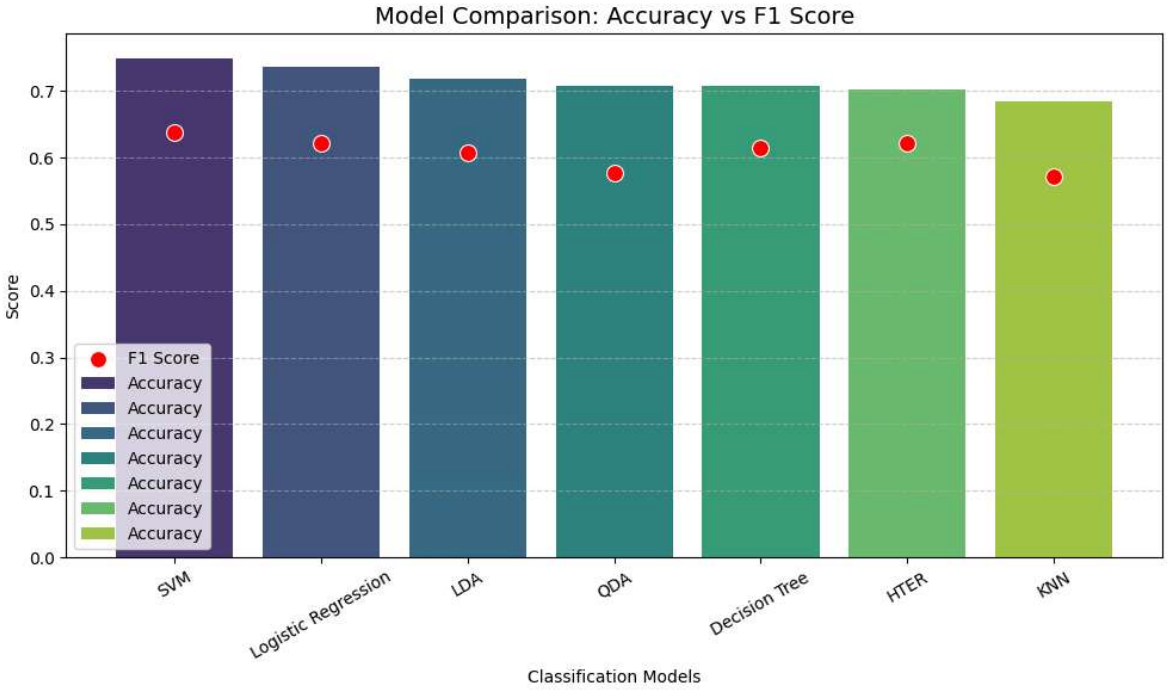
In [12]:

```
# Step 4B – Visualize performance comparison
import matplotlib.pyplot as plt
import seaborn as sns

# Sort by accuracy for better readability
results_df = results_df.sort_values(by="Accuracy", ascending=False)

plt.figure(figsize=(10, 6))
sns.barplot(x="Model", y="Accuracy", data=results_df, palette="viridis", label="
sns.scatterplot(x="Model", y="F1 Score", data=results_df, color="red", s=100, la
plt.title("Model Comparison: Accuracy vs F1 Score", fontsize=14)
plt.xlabel("Classification Models")
plt.ylabel("Score")
plt.legend()
```

```
plt.xticks(rotation=30)
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```




In [14]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticD
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

# Step 4: Compare multiple models
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'KNN': KNeighborsClassifier(),
    'LDA': LinearDiscriminantAnalysis(),
    'QDA': QuadraticDiscriminantAnalysis(),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'SVM': SVC(kernel='rbf', probability=True, random_state=42),
    'HTER': LogisticRegression(max_iter=1000)  # Placeholder for HTER until impl
}

model_results = {}  # 🧠 define here

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    model_results[name] = {
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall': recall_score(y_test, y_pred),
        'F1 Score': f1_score(y_test, y_pred)
    }

# Display summary table
import pandas as pd
```

```
results_df = pd.DataFrame(model_results).T.round(3)
display(results_df)
```

| | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| **Logistic Regression** | 0.737 | 0.578 | 0.673 | 0.622 |
| **KNN** | 0.684 | 0.507 | 0.655 | 0.571 |
| **LDA** | 0.719 | 0.552 | 0.673 | 0.607 |
| **QDA** | 0.708 | 0.540 | 0.618 | 0.576 |
| **Decision Tree** | 0.708 | 0.533 | 0.727 | 0.615 |
| **SVM** | 0.749 | 0.594 | 0.691 | 0.639 |
| **HTER** | 0.737 | 0.578 | 0.673 | 0.622 |

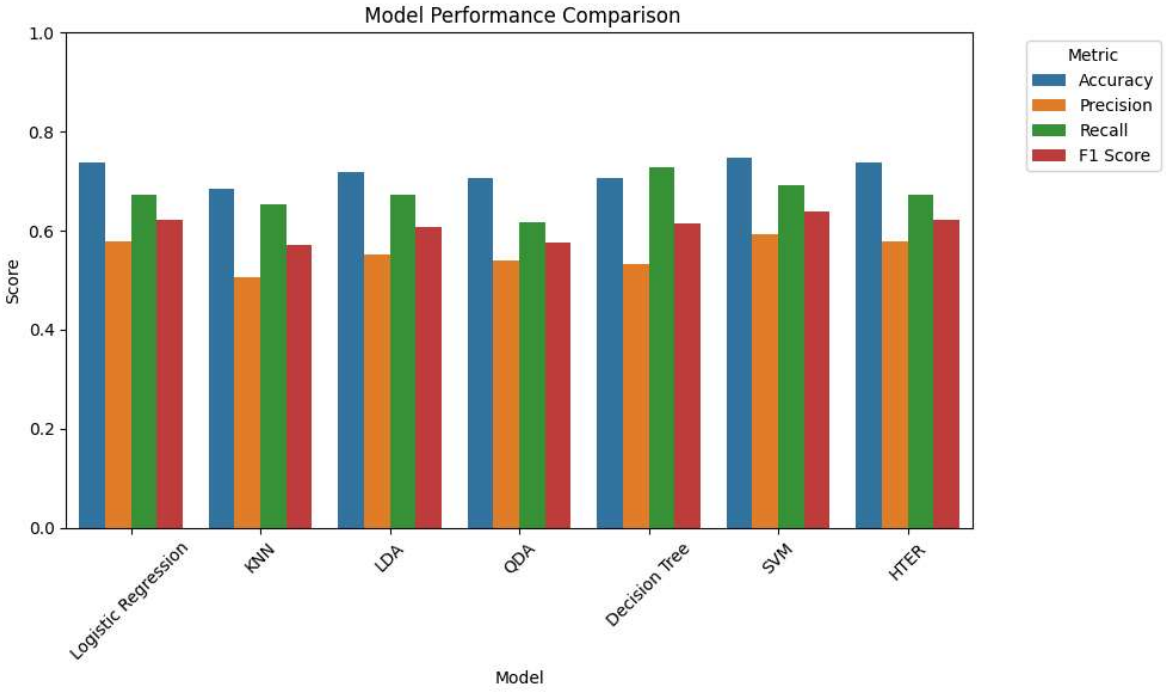
In [16]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Convert results dictionary to DataFrame
results_df = pd.DataFrame(model_results).T.reset_index()
results_df.rename(columns={'index': 'Model'}, inplace=True)

# Melt and visualize
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
results_melted = results_df.melt(id_vars='Model', value_vars=metrics, var_name='

plt.figure(figsize=(10,6))
sns.barplot(x='Model', y='Score', hue='Metric', data=results_melted)
plt.title(' Model Performance Comparison')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=45)
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

# Print best model
best_model = results_df.loc[results_df['Accuracy'].idxmax(), 'Model']
print(f"- Best model based on Accuracy: {best_model}")
```




- Best model based on Accuracy: SVM

In [17]:
```
## Compute Additional Metrics
from sklearn.metrics import roc_auc_score, log_loss, brier_score_loss, confusion
from sklearn.model_selection import cross_val_score, StratifiedKFold
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Initialize results storage
deep_results = []

# 5-fold stratified cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba

    # Compute metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba) if y_proba is not None else np.nan
    logloss = log_loss(y_test, y_proba) if y_proba is not None else np.nan
    brier = brier_score_loss(y_test, y_proba) if y_proba is not None else np.nan

    # Cross-validation accuracy
    cv_acc = cross_val_score(model, X, y, cv=cv, scoring='accuracy').mean()

    deep_results.append({
        'Model': name,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1 Score': f1,
```

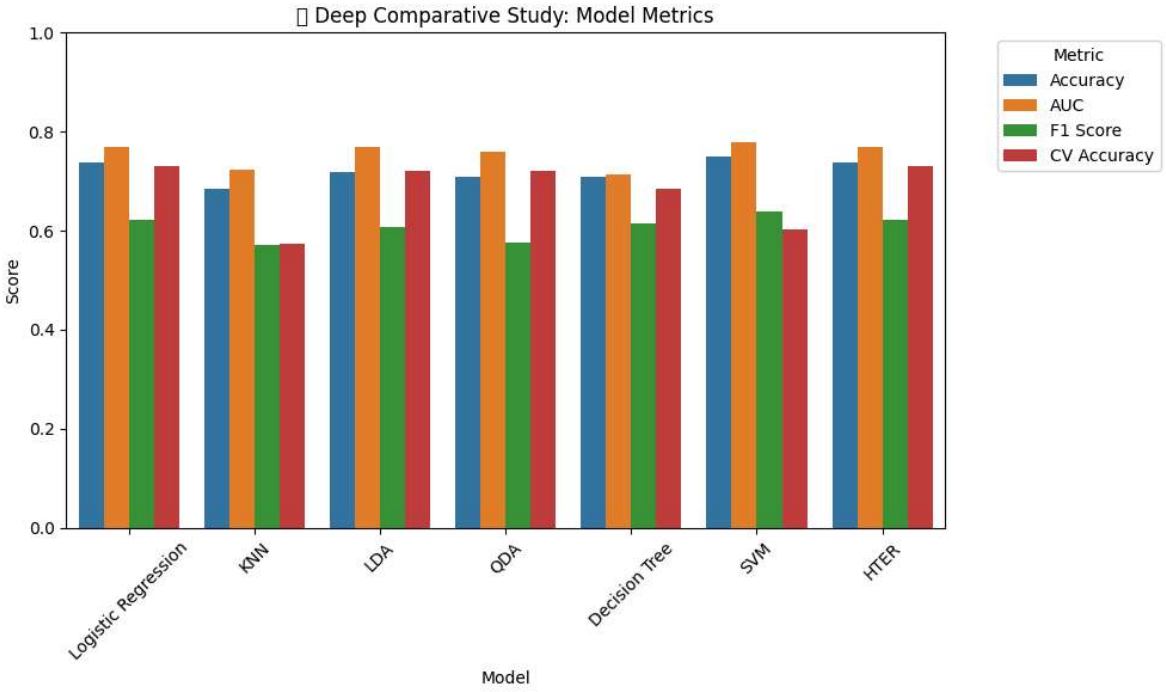
```
        'AUC': auc,
        'Log Loss': logloss,
        'Brier Score': brier,
        'CV Accuracy': cv_acc
    })

# Convert to DataFrame
deep_df = pd.DataFrame(deep_results).round(3)
display(deep_df)
```

| | Model | Accuracy | Precision | Recall | F1 Score | AUC | Log Loss | Brier Score | CV Accuracy |
|---|---|---|---|---|---|---|---|---|---|
| **0** | Logistic Regression | 0.737 | 0.578 | 0.673 | 0.622 | 0.768 | 0.544 | 0.181 | 0.730 |
| **1** | KNN | 0.684 | 0.507 | 0.655 | 0.571 | 0.723 | 1.775 | 0.215 | 0.573 |
| **2** | LDA | 0.719 | 0.552 | 0.673 | 0.607 | 0.769 | 0.546 | 0.182 | 0.721 |
| **3** | QDA | 0.708 | 0.540 | 0.618 | 0.576 | 0.759 | 0.614 | 0.197 | 0.721 |
| **4** | Decision Tree | 0.708 | 0.533 | 0.727 | 0.615 | 0.713 | 10.539 | 0.292 | 0.685 |
| **5** | SVM | 0.749 | 0.594 | 0.691 | 0.639 | 0.780 | 0.521 | 0.171 | 0.603 |
| **6** | HTER | 0.737 | 0.578 | 0.673 | 0.622 | 0.768 | 0.544 | 0.181 | 0.730 |

In [18]:
```
# Melt the DataFrame for plotting
metrics_to_plot = ['Accuracy', 'AUC', 'F1 Score', 'CV Accuracy']
deep_melted = deep_df.melt(id_vars='Model', value_vars=metrics_to_plot, var_name

plt.figure(figsize=(10,6))
sns.barplot(x='Model', y='Score', hue='Metric', data=deep_melted)
plt.title('🔍 Deep Comparative Study: Model Metrics')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=45)
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

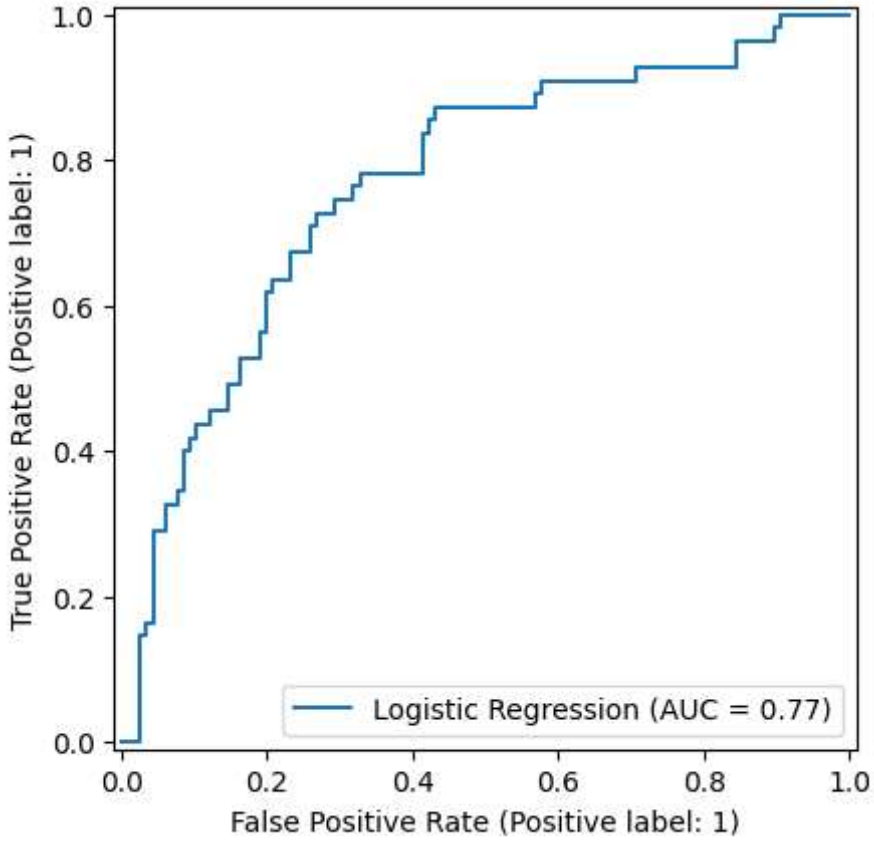


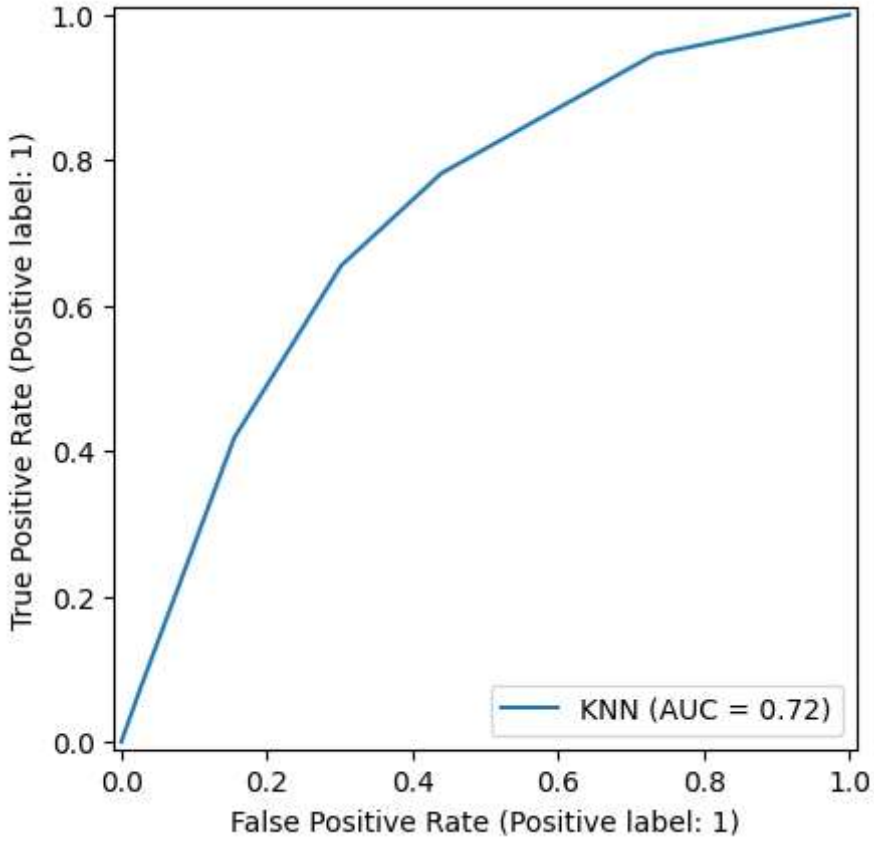

In [19]:
```
plt.figure(figsize=(7,6))

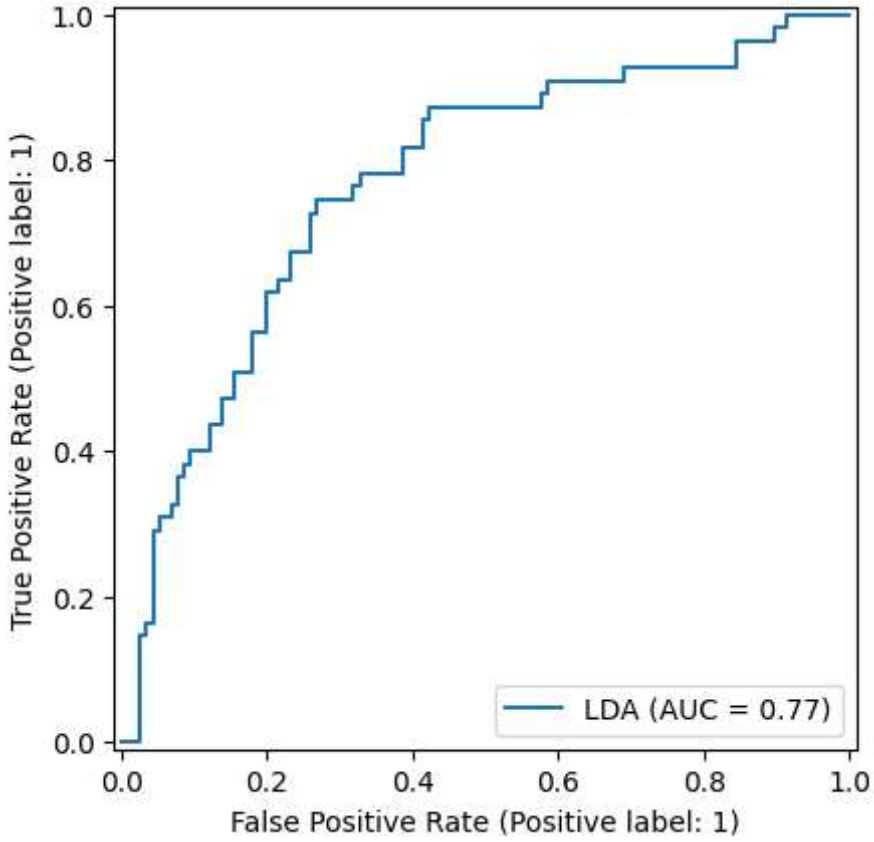
for name, model in models.items():
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_test)[:, 1]
        RocCurveDisplay.from_predictions(y_test, y_proba, name=name)

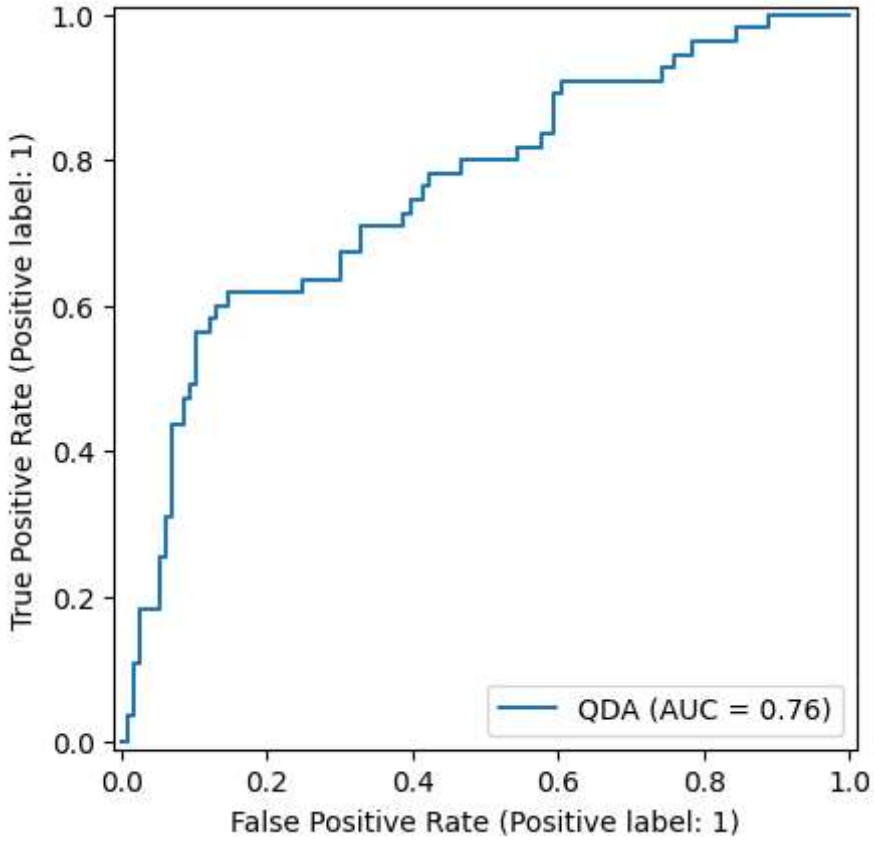
plt.title('📈 ROC Curves Comparison Across Models')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.legend()
plt.tight_layout()
plt.show()
```

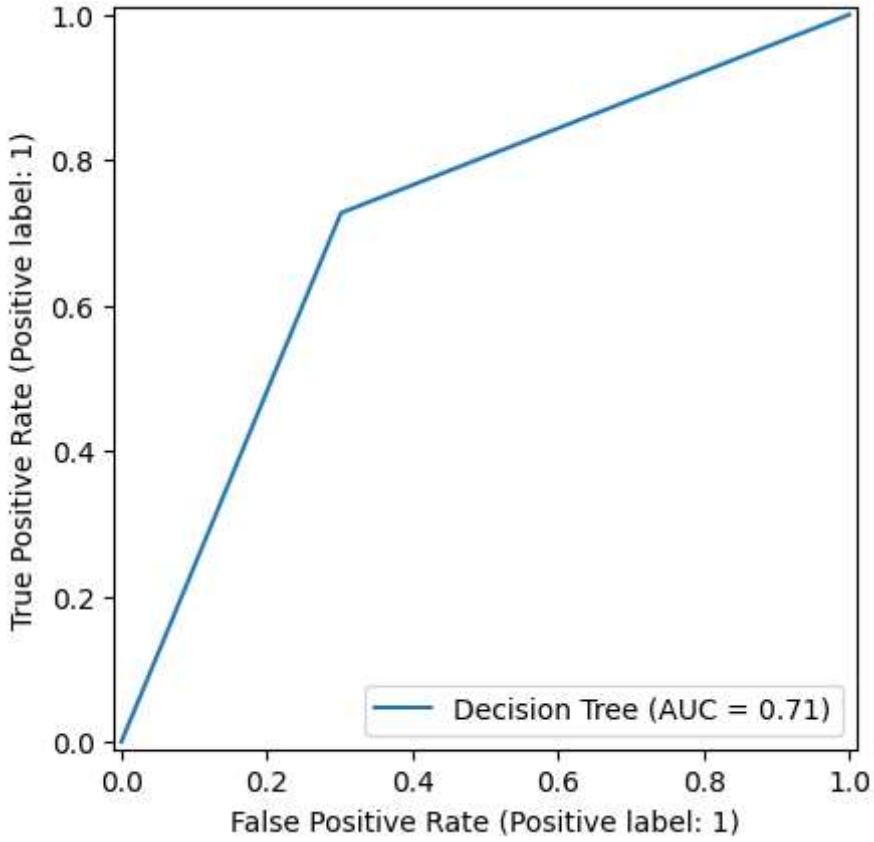
```
<Figure size 700x600 with 0 Axes>
```

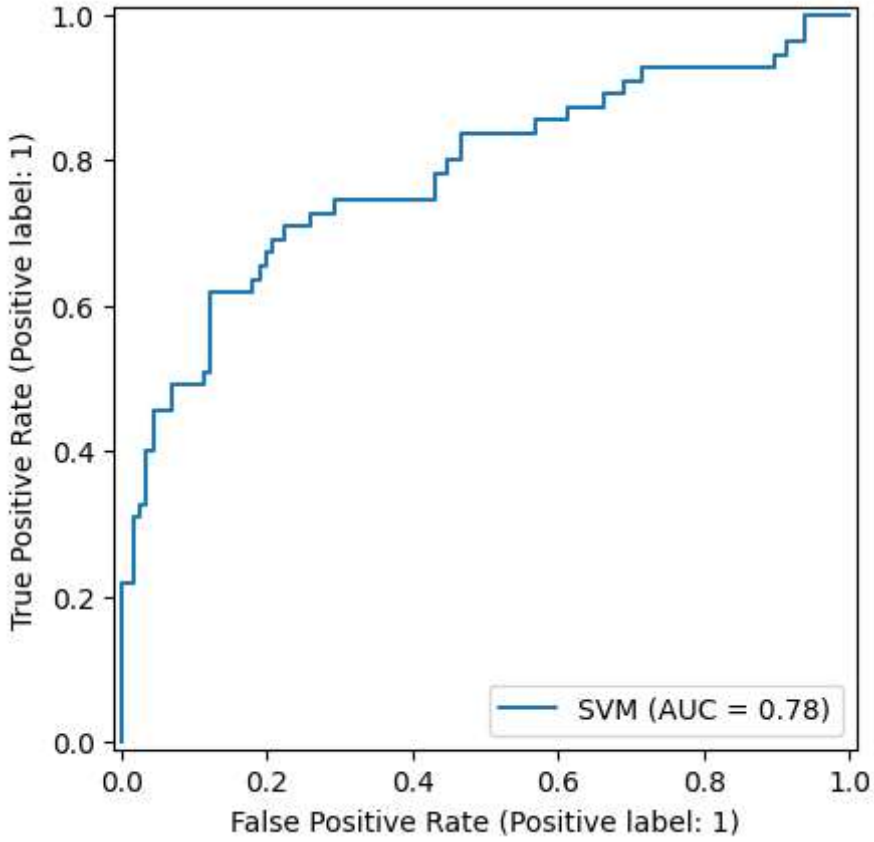

True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
Logistic Regression (AUC = 0.77)
0.0
0.2
0.4
0.6
0.8
1.0

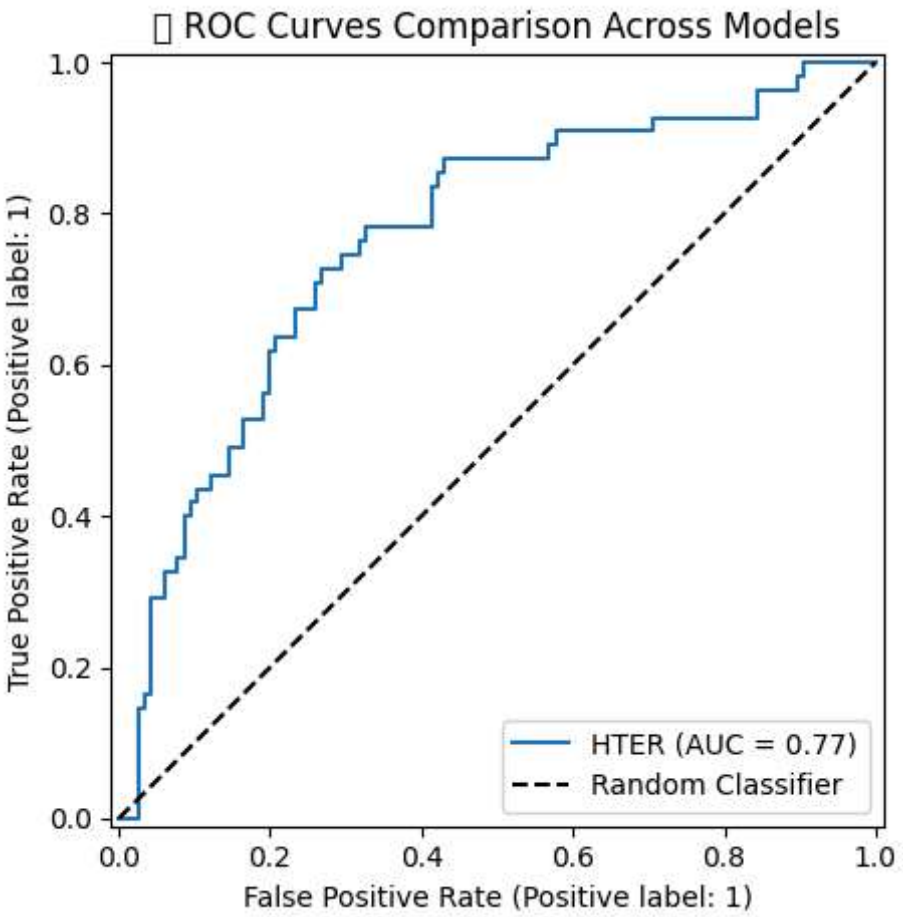

True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
KNN (AUC = 0.72)
0.0
0.2
0.4
0.6
0.8
1.0


True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
LDA (AUC = 0.77)
0.0
0.2
0.4
0.6
0.8
1.0


True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
QDA (AUC = 0.76)
0.0
0.2
0.4
0.6
0.8
1.0

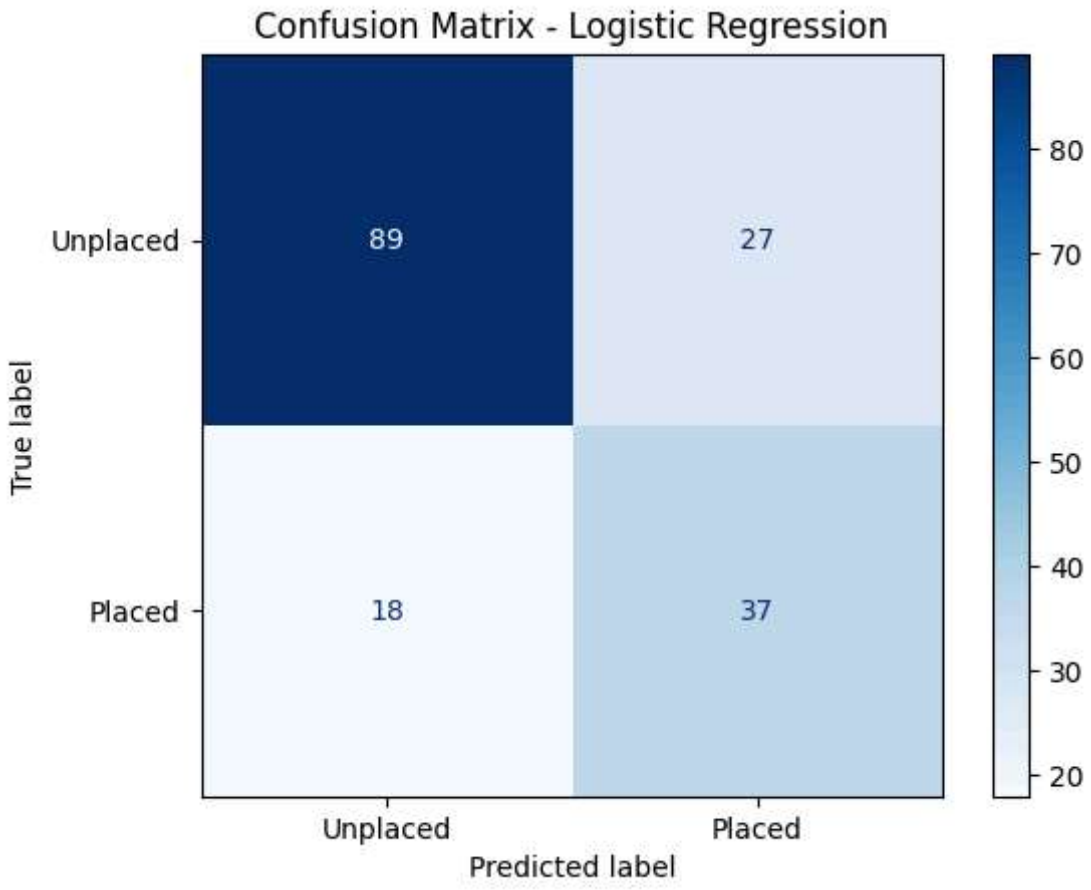

True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
Decision Tree (AUC = 0.71)
0.0
0.2
0.4
0.6
0.8
1.0

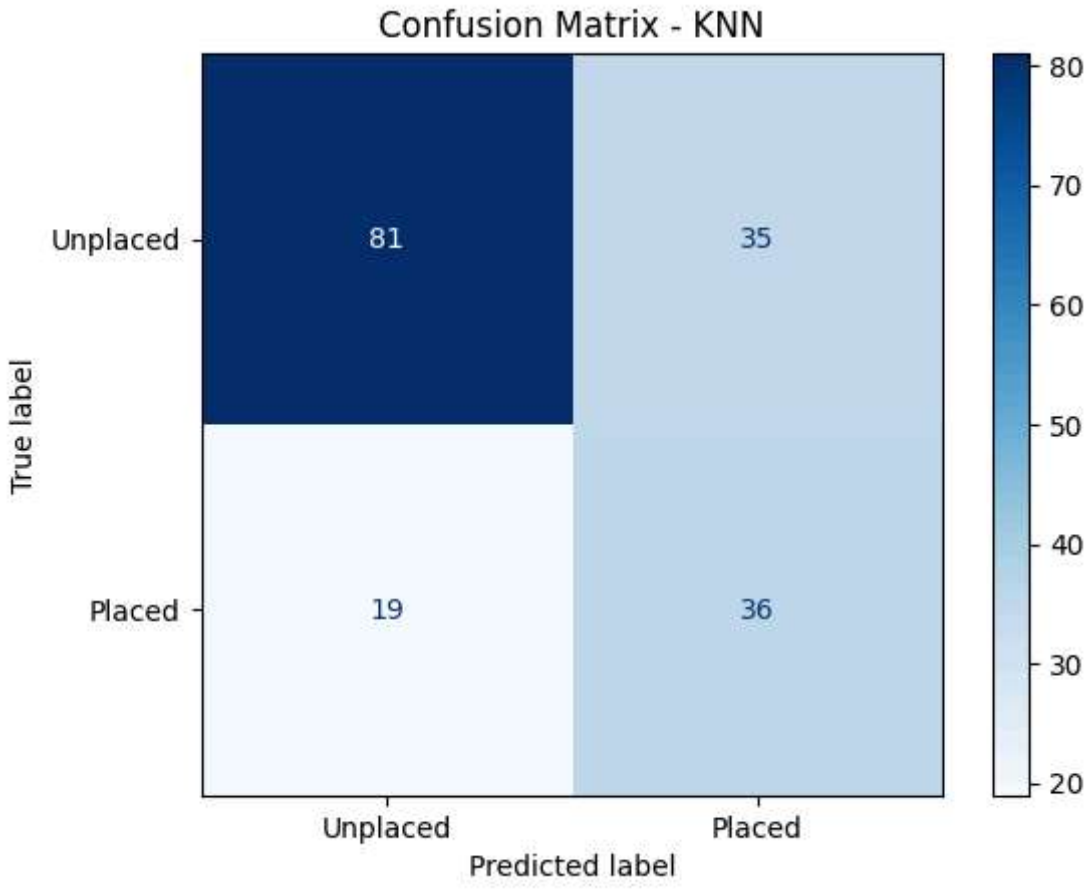

True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
SVM (AUC = 0.78)
0.0
0.2
0.4
0.6
0.8
1.0

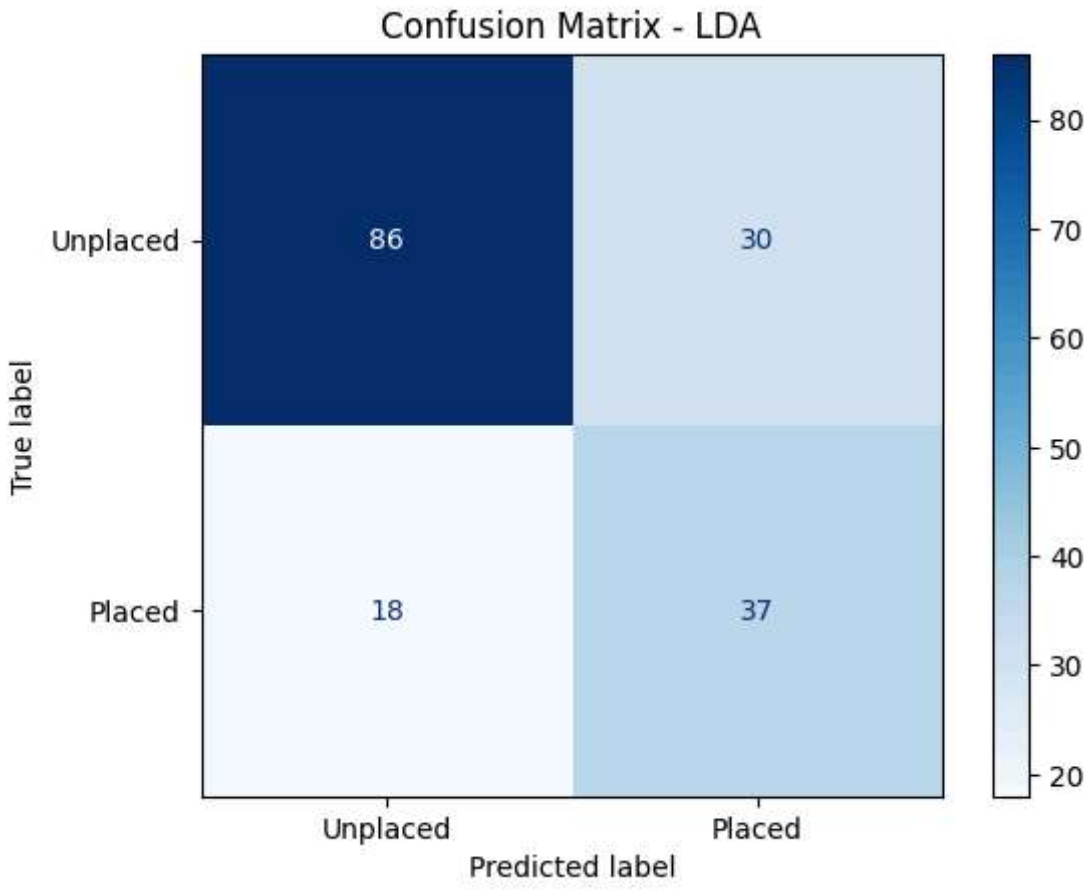


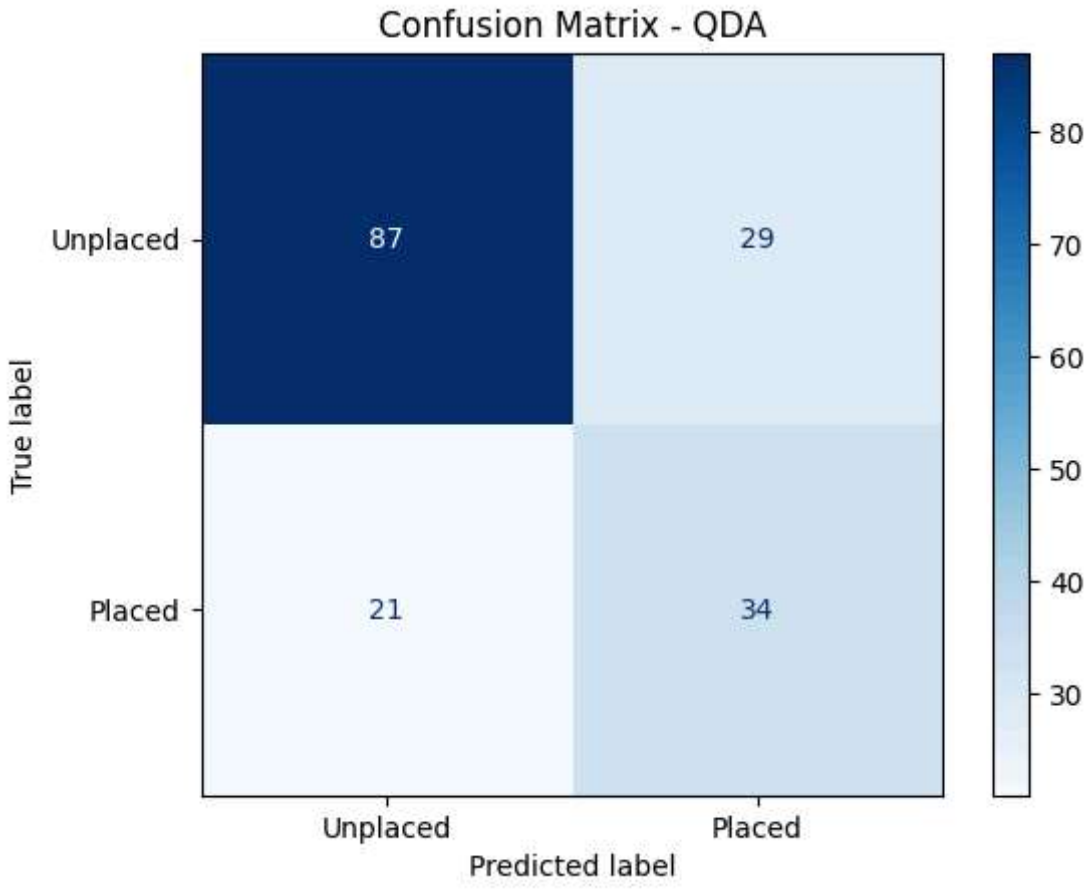

In [20]:
```
from sklearn.metrics import ConfusionMatrixDisplay

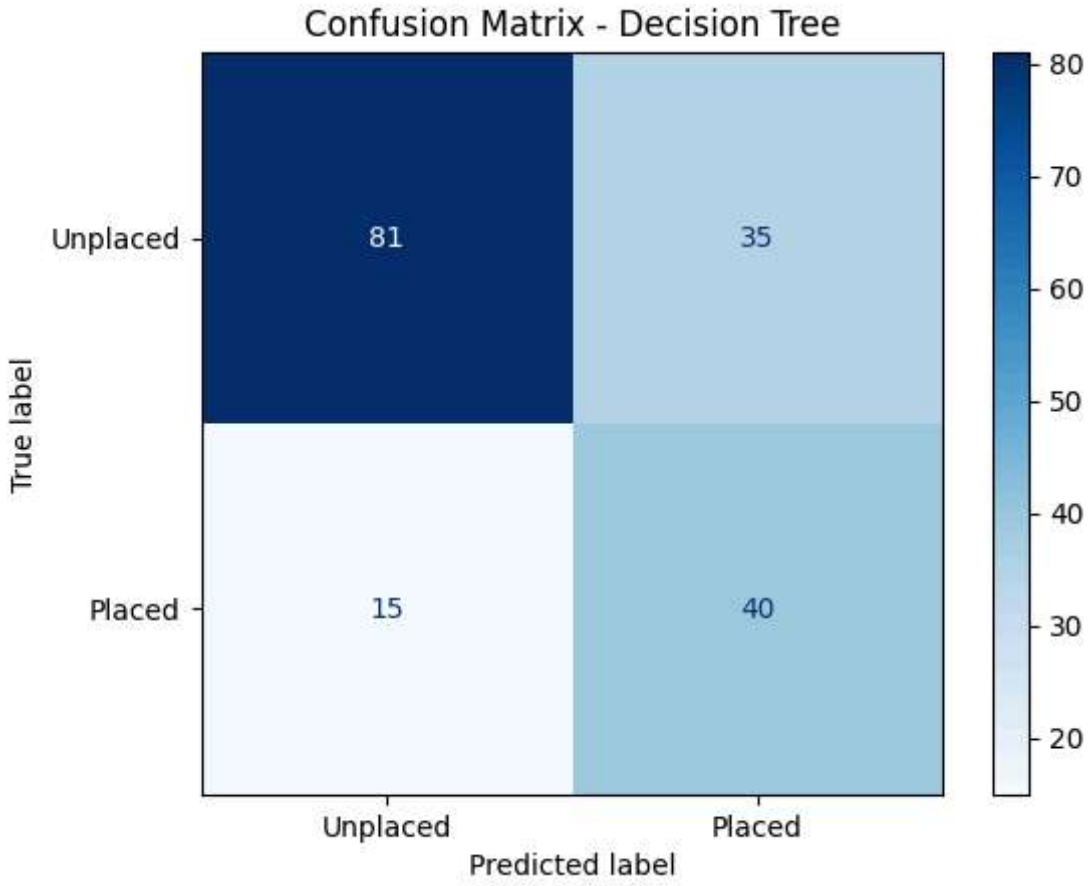
for name, model in models.items():
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Unplaced
    disp.plot(cmap='Blues')
    plt.title(f'Confusion Matrix - {name}')
    plt.show()
```

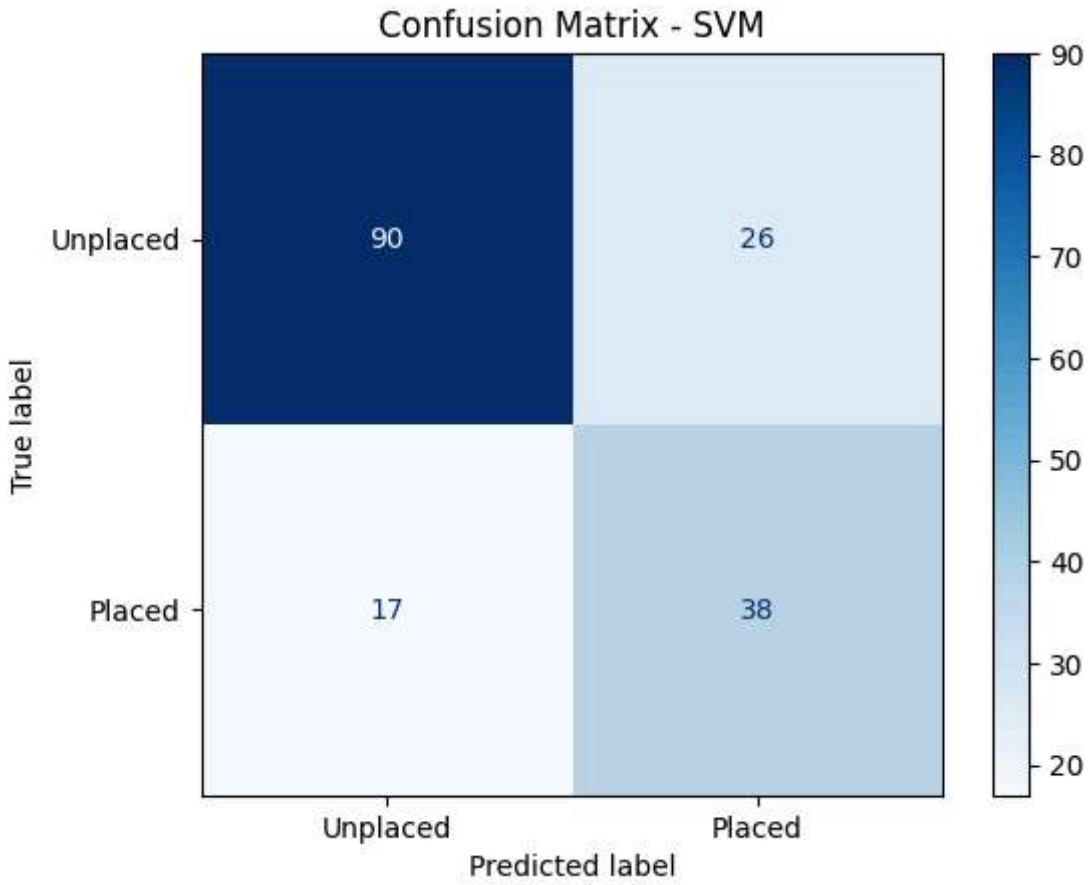

Confusion Matrix - Logistic Regression
Unplaced
Placed
True label
89
27
18
37
Unplaced
Placed
Predicted label
80
70
60
50
40
30
20

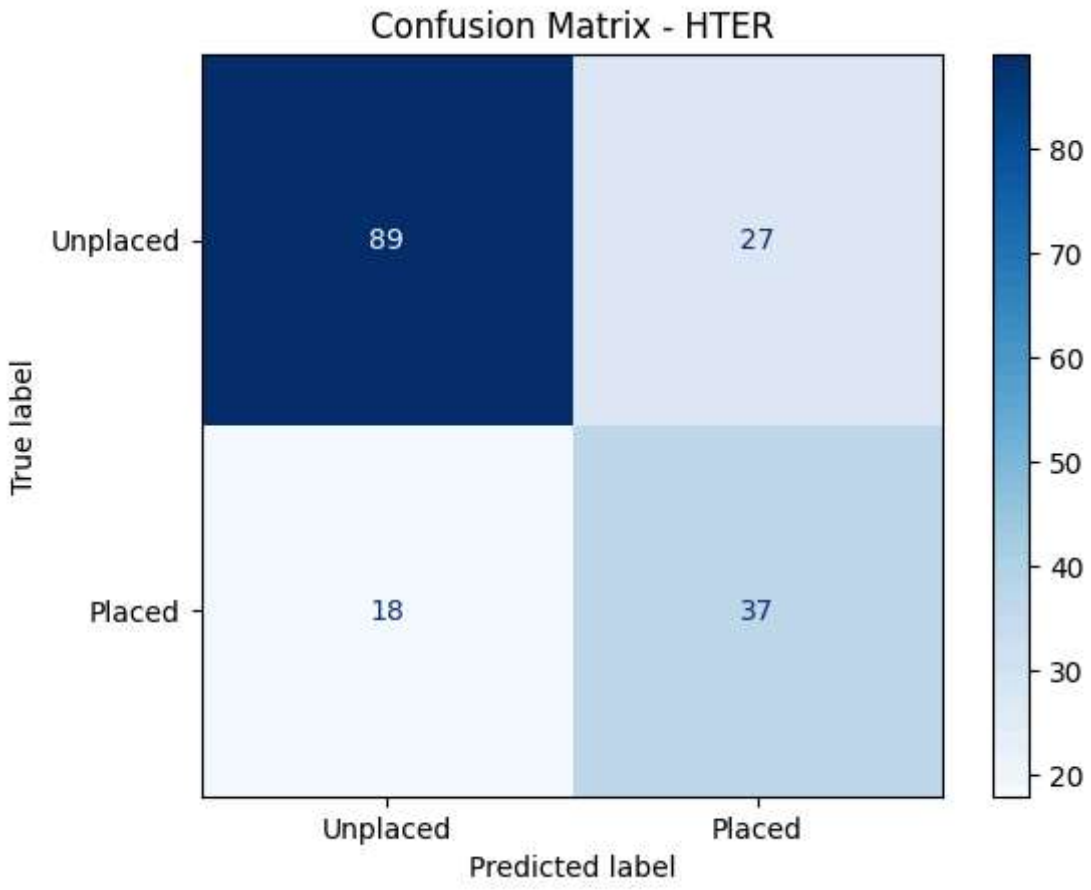

Confusion Matrix - KNN
Unplaced
Placed
True label
81
35
19
36
Unplaced
Placed
Predicted label
80
70
60
50
40
30
20


Confusion Matrix - LDA
Unplaced
Placed
True label
86
30
18
37
Unplaced
Placed
Predicted label
80
70
60
50
40
30
20


Confusion Matrix - QDA
Unplaced
Placed
True label
87
29
21
34
Unplaced
Placed
Predicted label
80
70
60
50
40
30


Confusion Matrix - Decision Tree
Unplaced
Placed
True label
81
35
15
40
Unplaced
Placed
Predicted label
80
70
60
50
40
30
20


Confusion Matrix - SVM
Unplaced
Placed
True label
90
26
17
38
Unplaced
Placed
Predicted label
90
80
70
60
50
40
30
20




In [21]:
```
## McNemar's Test (HTER vs Logistic Regression)
#This test examines whether the two classifiers make different numbers of errors
from statsmodels.stats.contingency_tables import mcnemar

# Get predictions from Logistic Regression and HTER
y_pred_log = models['Logistic Regression'].predict(X_test)
y_pred_hter = models['HTER'].predict(X_test)

# Create contingency table
b01 = np.sum((y_pred_log == y_test) & (y_pred_hter != y_test))  # Logistic corre
b10 = np.sum((y_pred_log != y_test) & (y_pred_hter == y_test))  # HTER correct,

table = [[0, b01], [b10, 0]]
result = mcnemar(table, exact=True)

print("🔷 McNemar's Test Result:")
print(f"Statistic: {result.statistic}, p-value: {result.pvalue:.4f}")
if result.pvalue < 0.05:
    print(" The difference between HTER and Logistic Regression is statistically
else:
    print(" No statistically significant difference between the models.")
```

```
🔷 McNemar's Test Result:
Statistic: 0.0, p-value: 1.0000
 No statistically significant difference between the models.
```

In [ ]: